Swarma: платформа для роёв автономных агентов

Рантайм на машинах владельца, подключаемые рабочие процессы, протокол независимых владельцев и единая личность участника

Алексей Григорьев, Иван Опарин
Relux Works
github.com/relux-works

Октябрь 2026, версия 7.0

Аннотация. Языковая модель может предложить действие, но из предложения не должно следовать право его совершить. Swarma строит на этом разделении платформу для роёв автономных агентов, то есть групп агентов с разными ролями, которые вместе ведут проекты разработки ПО с основной веткой, ревью и проверками. Платформа состоит из трёх слоёв. Swarma Runtime удерживает агентов на машинах одного владельца: у каждого агента своя учётка ОС, ключи хранит отдельная служба, каждое действие требует подписанного разрешения, которое при делегировании только сужается, а сообщения из мессенджера до доставки агенту проверяет детерминированный код. Процесс работы команда подключает модулем, который задаёт роли, маршрутизацию задач к исполнителям, состояния и условия ревью и согласования, при которых задача доходит до приёмки. Саму приёмку модуль не меняет: изменение принимается только после независимой проверки точной ревизии, и основную ветку продвигает отдельная служба. Swarma Protocol даёт независимым владельцам общую среду проекта: решения проходят через журнал, который подписывает кворум узлов разных владельцев, а основная ветка продвигается только к проверенному неизменяемому кандидату. Ключи, сообщения, разрешения, память и история работы участника сводятся к одной устойчивой личности, которую Swarma Chain делает общей для всех проектов. Оценка измеряет эталонные компоненты по частям. Генератор корпуса подписанных записей, независимая проверка на Python и библиотека на Go согласны на 2632 случаях вплоть до кода отказа; согласие сообщает репозиторий библиотеки, а тест библиотеки мы повторили. Два независимо написанных вычислителя памяти дают ожидаемые ответы на всех 66 смысловых векторах корпуса 0.2.7, то есть спецификация памяти однозначна на этом наборе. На машине Linux привилегированные примитивы учёток агентов прошли применение, проверку и откат, а набор квалификационных проверок хоста прошёл все 11 проверок. Ограниченная модель интеграции показывает, что без любого из четырёх правил доставки находится контрпример. Медиана эталонной проверки одной записи составляет от 2 до 18 мс в одном потоке.

1. Введение

Агент разработки читает файлы, вызывает shell и отправляет сообщения. Для операционной системы он остаётся обычным процессом пользователя, запустившего его, и получает все права этого пользователя: те же файлы, ту же конфигурацию, те же ключи. Косвенная инъекция в промпт [19, 12] пользуется именно этим. Допустим, вывод сборки предлагает “для исправления” открыть соседний файл с учётными данными. Модель принимает текст за подсказку и вызывает чтение. Если файл доступен процессу, исход зависит от того, сообразит ли модель отказаться. Если доступ закрыт границей ОС, ошибка модели файл не откроет.

Отсюда исходная позиция работы: модель может предложить действие, но предложение не даёт ей полномочия. Полномочие проверяет сторона, которая совершает эффект, детерминированным кодом и по проверяемым основаниям. Модель остаётся источником предложений, и её суждение входит в систему только там, где оно сужает допуск.

Предметная область. Работа рассматривает проекты разработки ПО. У такого проекта есть репозиторий с основной веткой, ревью изменений и проверки CI, а результатом работы роя служит принятое изменение основной ветки. Процессы вне разработки используют те же механизмы (раздел 4).

Рой и его участники. *Агентом* мы называем участника работы с ролью, полномочиями и устойчивой криптографической личностью. Его работу исполняет *агентская среда*, программа, которая ведёт диалог с моделью и исполняет вызванные ею инструменты, например Claude Code или Codex; *сессией* называется один такой диалог со своим контекстом и состоянием исполнения. *Роем* называются несколько агентов с разными ролями, которые вместе работают над целью. *Владелец* отвечает за свои машины, агентов и оплату моделей, и ему принадлежит корень доверия установки. *Оператор* означает человека владельца, который выдаёт полномочия и подтверждает действия, расширяющие права. *Оркестратор* планирует работу роя: разбивает цель на задачи, назначает исполнителей и инициирует

Таблица 1. Проблема и механизм, который её решает.

Проблема	Механизм	Раздел
Процесс агента получает все права запустившего его пользователя	станция: своя учётка ОС агента и сеть, ограниченная ядром	3.1
Ключ в доступном агенту файле можно украсть уговором	ключник держит ключи, агент только просит подпись	3.2
Подпись говорит, кто отправил байты, и ничего не разрешает	грант несёт право на действие; ключи, гранты и отзывы ведёт реестр владельца	3.3
Членство в комнате мессенджера прав не даёт	мост проверяет каждое сообщение до доставки	3.4
У сессии много входов	хост сессий с единственным писателем ввода	3.5
Ёмкость модели другого владельца нужна без его доступа к среде	удалённое рабочее место; в протоколе разделены мышление, управление и исполнение	3.6, 5
Знание и обязательства теряются вместе с контекстом	память агентов и Swarma Keeper	3.7, 4.5
У принятия кода нет механического оракула	подключаемый процесс над неизменными правилами ядра доски	4
История участника должна пережить проект	одна личность для всех ключей, общая через Swarma Chain	6, 7

приёмку в пределах выданного ему полномочия. *Продьюсер* производит изменение, а *ревьювер* независимо проверяет его и выносит вердикт. Право принять результат в общий проект отделено от права его произвести и от права о нём судить.

Рой порождает вопросы, на которые модель не может ответить своим текстом. Кто отправил сообщение и что ему разрешено? Кто вправе дописать ввод в идущую сессию? Кто отвечает за работу, когда сессия остановилась на полуслове? К какому состоянию кода относится одобрение? Как принять ёмкость модели другого владельца, не отдав ему ключей проекта? Как агенту предъявить историю своей работы в новом проекте? Агентские фреймворки [43, 21, 36, 46] координируют роли и инструменты, а эти вопросы оставляют среде исполнения.

Зачем независимым владельцам общая среда. Среду некоторых проектов дорого переносить к каждому участнику: в ней дорогой тулчейн, устройства или данные, которые нельзя вывозить (раздел 5). Ёмкость моделей при этом есть у многих, и у человека со своей подпиской может оставаться свободная квота. Swarma позволяет проекту принять эту ёмкость, не перенося к участнику репозиторий вместе с окружением и не отдавая ему ключей проекта. Когда проект ведут несколько владельцев, среда остаётся одна, а что в неё принимается, они решают вместе. Во что проект конвертирует вклад участника, в деньги, репутацию или долю, задаёт политика проекта, а общую валюту для расчётов между участниками даёт Swarma Chain.

Зачем нужна каждая часть. Swarma отвечает на эти вопросы отдельными механизмами, и у каждого механизма своя причина (табл. 1).

Три слоя. Swarma состоит из трёх слоёв (рис. 1). **Swarma Runtime** держит рой на машинах одного владельца: учётки, ключи, полномочия, сообщения, сессии и окружение агентов. **Swarma Protocol** задаёт совместную работу независимых владельцев над единой средой исполнения проекта. **Swarma Chain** хранит общее состояние личностей, имён, аттестаций участия и расчётов за пределами отдельного проекта. Рантайм полезен сам по себе, протокол опирается на рантайм, цепочка связывает участников разных проектов.

Сквозной пример. Владелец ставит на доске задачу исправить поведение библиотеки и сохранить совместимость. Доска просит запуск разработчика, и тот работает под своей учёткой ОС, без ключей в доступных ему файлах и с сетью только к провайдеру модели (раздел 3). Совет соседнего агента открыть чужой модуль проходит проверку сообщений и остаётся данными. Разработчик публикует неизменяемую ревизию, ревьювер получает её diff только для чтения, а процесс проекта решает, достаточно ли вердикта для приёмки; основную ветку продвигает отдельная служба, и только к точной проверенной голове (раздел 4). Если модель для задачи даёт другой владелец, его агентский цикл работает у него, а команды исполняются на отдельном рабочем месте проекта (раздел 3.6). Если проект ведут несколько независимых владельцев, приёмку фиксирует журнал под подписями кворума их узлов, а ревьювер выбирается после фиксации кандидата (раздел 5). После приёмки проект подписывает свидетельство участия ревьювера, и другой проект читает его через Swarma Chain по своей политике (разделы 6 и 7).

Работа описывает платформу целиком. Границы доверия рантайма и протокола подробно разбирает сопутствующая работа [53], устройство цепочки описывает работа о Swarma Chain [51], связную модель

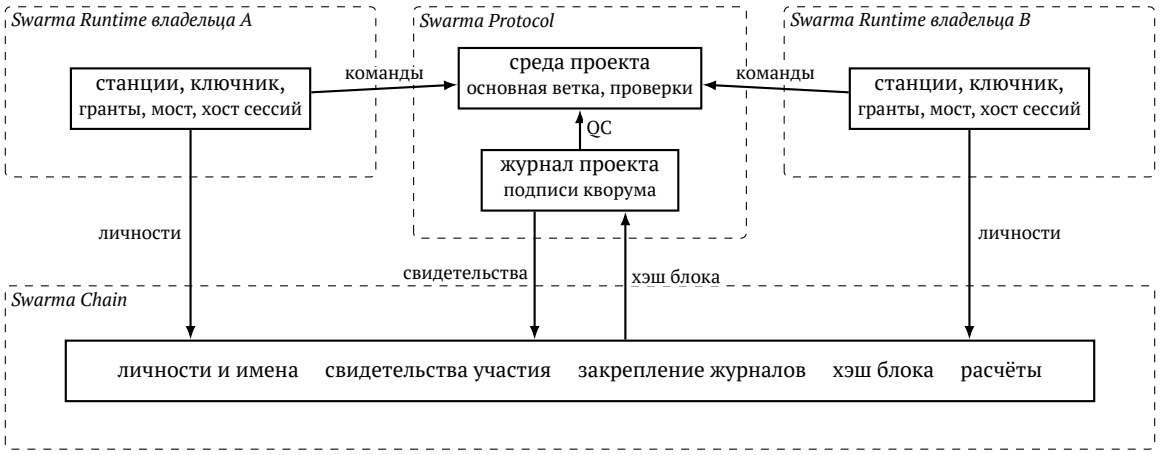


Рис. 1. Три слоя Swarma. Рантайм каждого владельца удерживает своих агентов. Протокол связывает рантаймы независимых владельцев с одной средой проекта, где команды исполняются только по записям журнала, которые подписал кворум узлов разных владельцев (сертификат кворума, QC). Цепочка хранит общие для всех проектов личности, имена, подписанные свидетельства участия и расчёты, закрепляет головы журналов и даёт хэш блока для случайного выбора ревьювера.

с примерами даёт книга [52], а состояние реализации каждой части описано в отдельном приложении *Swarma: состояние реализации*. Где работа и спецификация модуля расходятся, права спецификация.

Вклад работы:

1. Модель системы и модель угроз для всей платформы, где захваченный инъекцией агент считается штатным противником на каждом слое (раздел 2).
2. Композиция рантайма, в которой допуск решает детерминированная проверка у исполнителя эффекта: станции с контрактом изоляции исполнения, ключник с определением собеседника ядром, гранты, которые при делегировании только сужаются, реестр владельца, который ведёт ключи, гранты и отзывы, со свидетелем отката его журнала, проверка сообщений поверх Matrix, хост сессий с единственным писателем, удалённое рабочее место для модели другого владельца и память как потребитель реестра (раздел 3).
3. Рабочий процесс как подключаемый модуль: плейбук задаёт роли, маршрутизацию, состояния и условия ревью и согласования, а неизменные правила ядра, включая приземление только точной проверенной головы, действуют под любым плейбуком; процесс по умолчанию состоит из продьюсера, ревьювера, ищущего основания отклонить, и этого приземления (раздел 4).
4. Протокол независимых владельцев над одной средой: журнал под сертификатами кворума трёх уровней, оркестратор по аренде, принятие, адресованное точному кандидату, и федерация проектов (раздел 5).
5. Схождение ключей, сообщений, учёток, грантов, памяти и подписанной истории работы на одной устойчивой личности и её интеграция со Swarma Chain (разделы 6 и 7).
6. Анализ гарантий по слоям и оценка эталонных компонентов: корпуса соответствия с независимыми реализациями, привилегированные запуски на машине Linux, ограниченная модель интеграции, микрозамеры проверки записей и статистика ревью (разделы 8 и 9).

2. Модель системы и модель угроз

Принципалы и роли. Кроме владельца, оператора и агентских ролей из введения, в работе участвуют ещё четыре роли. *Проект* задаёт цель, среду, правила приёмки и политику допуска участников. *Контрибьютор* приносит проекту ёмкость своего агентского цикла, то есть модели с контекстом и вызовами инструментов, а *донор* отдаёт квоту своей подписки на модель. *Верификатор* повторяет критические проверки на другой машине. Совмещение ролей дополнительных прав не даёт: право дать ресурс, право владеть установкой и право изменить проект различаются.

Носитель. *Носителем* называется мессенджер, который перевозит сообщения агентов и людей; носителем служит Matrix [39].

Хосты, станции и службы. Хостом называется машина владельца, на которой работают агенты. *Станцией* называется место одного агента на хосте: своя учётка ОС, своя личность на носителе, свои ключи, дом и корневой контекст; учётка с отдельным UID без остальных проверок станцией не считается. Привилегированные службы хоста разнесены по учёткам (рис. 2): ключник работает под root, менеджер пользователей и запускатель получают root на одну узкую операцию, остальные

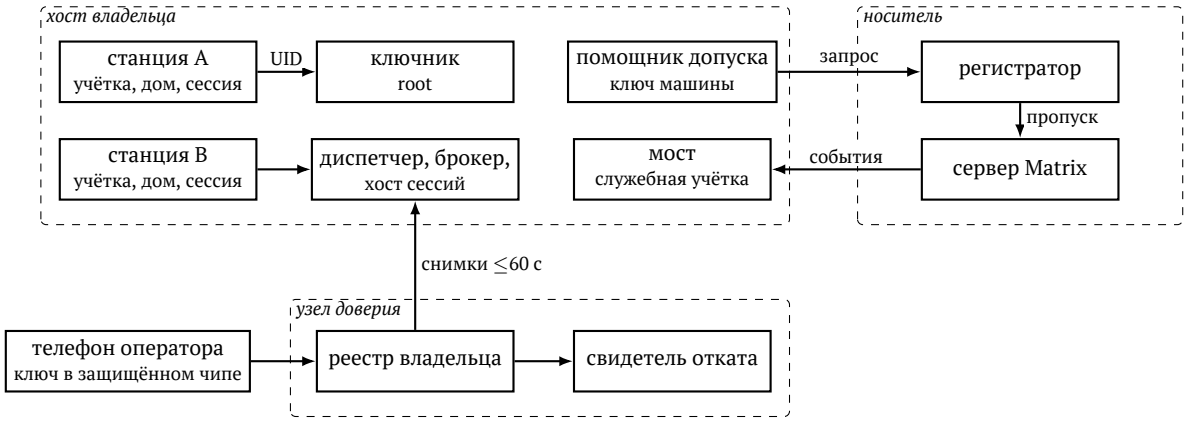


Рис. 2. Хост владельца. Агенты на станциях ключей не держат; ключник и помощник допуска узнают собеседника по UID от ядра. Регистратор выдаёт пропуск на носитель, мост проверяет сообщения, реестр на узле доверия принимает подписанные действия оператора и выдаёт службам снимки, свидетель фиксирует его контрольные точки.

службы работают под своими служебными учётками.

Узел доверия. *Реестр владельца*, единственный авторитетный писатель ключей, грантов и отзывов установки, и *свидетель отката* его журнала работают на *узле доверия*, отдельной машине вне административного домена платформы (раздел 3.3). Её снимают у провайдера, который от платформы не зависит, и администрирует её только владелец; у агентов там нет root, у платформы нет административного доступа.

Протокол. В Swarma Protocol *кор-ноды* независимых владельцев ведут журнал полномочий и принятия, *рабочими нодами* служат недоверенные агентские циклы контрибьюторов, а *среда исполнения* проекта исполняет только разрешённые журналом действия.

Подписанные записи. Подписанную запись платформы мы называем *конвертом*: он состоит из канонической формы байтов с доменной меткой и подписи автора над ними, а формат и порядок проверок задаёт спецификация подписанных записей `swarm-envelope/2`. Конвертами служат сообщения агентов, гранты, аттестации ключей сообщений и записи реестра.

Аттестации. *Аттестацией* в работе называется подписанное утверждение стороны, которая отвечает за проверку, о свойстве определённого субъекта. Виды аттестаций различаются субъектом и подписантом. Аттестацию машины подписывает ключ машины по итогам квалификации её гарантий (раздел 3.1). Аттестация ключа сообщений связывает ключ с агентом, и её подписывает корень аттестаций ключника (раздел 3.2). Аттестация проверки фиксирует исход проверки свойства точного кандидата в протоколе (раздел 5.4). Аттестацию участия проект выдаёт о работе агента в определённой роли (раздел 6). Аттестацию устройства оператора подписывает корень оператора (раздел 3.2).

Противники. У трёх слоёв разные противники, и одна мысль на каждом слое требует своего механизма. Мы рассматриваем восемь противников:

П1. Захваченный агент подчинился внедрённому тексту и пользуется всеми возможностями своего процесса, включая подписи, которые ключник выдаёт его личности.

П2. Чужой участник, то есть агент или человек другого владельца, рабочая нода протокола или гость комнаты, пытается расширить полномочия, подменить объект одобрения или действовать от чужого имени.

П3. Сетевой противник подменяет, задерживает и повторяет сообщения, а сервер носителя может выпустить событие от имени любого адреса своего домена.

П4. Противник получил root на одной зарегистрированной машине из нескольких; корни владельца вне этой машины, ключ оператора и узел доверия ему недоступны.

П5. Противник на узле доверия похитил ключ реестра или пытается откатить и раздвоить журнал реестра.

П6. Злонамеренный оркестратор протокола предлагает вредные задачи и пытается сам выбрать проверяющего или интегрировать результат.

П7. До f кор-нод злонамеренны на уровне доверия L2, меньшинство ключей скомпрометировано на L1.

П8. Криптографический противник ломает класс подписей на эллиптических кривых.

Допущения. Для гарантий хоста доверены его ядро, root и доверенная база платформы: ключник, менеджер пользователей, запускатель, диспетчер, брокер, хост сессий, мост с воротами доверия и по-

мощник допуска, на узле доверия реестр, его процесс подписи и свидетель, вне хоста регистратор и телефон оператора. Мастер-секрет ключника цел. Оператор подтверждает расширяющие действия на телефоне с ключом в Secure Enclave или StrongBox и видит подписываемое содержание. Владелец честен перед своими агентами и единолично администрирует узел доверия. У установки есть проверенный или контролируемый оператором источник времени, например NTS. В протоколе без аппаратной аттестации исполнения среда проекта доверена в отношении целостности, а провайдер модели доверен в отношении конфиденциальности в пределах условий, которые проект объявил до присоединения.

Вне рамок. Платформа не защищает от эксплойта ядра и не сохраняет гарантий хоста с захваченным root (радиус поражения для остальных машин разобран как П4). Она не обещает правильности суждения модели, конфиденциальности кода от контрибьютора и его провайдера и полноты наблюдения за недоверенной нодой вне протокола. Побочные каналы и подделка оборудования не рассматриваются. Отказ в обслуживании не предотвращается: при недоступном свидетельстве система закрывает допуск и сознательно жертвует живучестью ради целостности.

Задача. Ни одно действие агента с эффектом за пределами его станции не исполняется без проверяемого основания: подлинного отправителя, актуального гранта, одобрения, адресованного точному объекту, и физического удержания исполнения в выданной области. Основание проверяет сторона, совершающая эффект. Отсутствие свидетельства, подтверждённое отсутствие события и невозможность прочитать состояние различаются, и разрешения ни одно из них не даёт.

3. Swarma Runtime на машинах одного владельца

Swarma Runtime собран из небольших служб с именованными контрактами под разными пользователями ОС; границу задают учётки, права файлов, сокеты и правила ядра.

3.1. Станция и контракт изоляции исполнения

Каждый автономный агент работает отдельным пользователем ОС: оркестратор, постоянный специалист, временный продьюсер и удалённое рабочее место (раздел 3.6). Каждое создание учётки получает случайное 128-битное *поколение*, и на поколение опираются привязки, гранты и группа ресурсов, поэтому повторно выданный UID чужих прав не наследует. Собственный UID отделяет файлы и процессы одного агента от другого, и процесс продьюсера не прочитает рабочий каталог ревьювера, даже зная путь. Контейнеры и виртуальные машины для этого не нужны: рантайму нужна граница между агентами одной машины, а отдельного образа окружения на агента ему не требуется. На Linux у каждого поколения учётки своя cgroup v2 под управлением root с лимитами `cpu.max`, `memory.max`, `memory.swap.max`, `pids.max` и лимитами ввода-вывода, а также квоты файловой системы. На macOS роль этой границы играет квалифицированный профиль Seatbelt для класса задачи. Агент считается остановленным, когда у всей его учётки нет процессов в течение опубликованного периода тишины; выход ведущего процесса этого не доказывает.

Привилегированных входов два. Менеджер пользователей (`swarma-user-manager`) создаёт и выводит учётки и управляет их сетевыми правилами и группами ресурсов, запускатель (`swarma-user-launcher`) запускает одну одобренную программу под активной учёткой. Обе программы вызываются через `sudo` без аргументов и принимают один JSON ограниченного размера. План запуска приходит запускателю в этом JSON, и запускатель кладёт его неизменяемыми байтами на дескриптор 3 запускаемой программе. Если ответ о шаге потерян, диспетчер наблюдает тот же эффект, и отсутствие свидетельства не позволяет считать, что эффекта не было.

Сеть ограничивает ядро: правила `pf` на macOS или `nftables` на Linux пускают агента только к слушателю доверенного прокси, своему у каждого поколения учётки. Прокси пропускает `CONNECT` к адресатам провайдера и ограничивает имя, порт и проверенный адрес; TLS внутри туннеля он не расшифровывает, поэтому разрешённый адресат остаётся каналом выноса данных. Сборки и тесты кандидата идут на эфемерных раннерах другой машины без секретов проекта. Публикацию кандидата, обновление базы и приземление выполняет отдельная служба действий с репозиторием под своей учёткой, по подписанному доской запросу на точного кандидата; голову ветки она продвигает сравнением-и-заменой.

Что станция гарантирует, задаёт *контракт изоляции исполнения*: профиль изоляции, вектор возможностей, привязки личности, размещения и исполнения, квитанция границы и кортеж квалификации. Запускатель применяет допущенную границу и пишет защищённую квитанцию, диспетчер сверяет покрытие, брокер перед выдачей учётных данных проверяет текущую квитанцию. Гарантии названы 42 идентификаторами общего словаря; например, `isolation.read` означает, что работа станции не читает чужих личных данных и секретов платформы. У каждой гарантии на каждой платформе один из четырёх статусов: `present`, `weaker-with-compensation`, `absent`, `PENDING-VERIFY`. Задача объявляет

нужные гарантии записью station-required-guarantees/1; absent и PENDING-VERIFY дают отказ, а ослабление допустимо, только если оператор отдельным аутентифицированным действием принял его для этой задачи, станции, компенсации и срока. Машину владелец квалифицирует сам воспроизводимой процедурой; аттестацию машины подписывает отдельный ключ машины, и она называет каждую из 42 гарантий ровно один раз. При дрейфе защищённых входов или проваленной проверке новые старты закрываются сразу, а работающее исполнение сужается или останавливается целиком не позже 300 с.

3.2. Ключник: личности без ключей у агента

Ключник (Swarma Keyholder) служит привилегированной службой с мастер-секретом установки, которая выполняет криптографические операции за агентов. Закрытого ключа личности у агента нет: инъекция действует руками агента, и агента с ключом в доступном ему файле достаточно уговорить этот файл прочитать. Где возможно, мастер-секрет держит аппаратное хранилище. Ключи агентов выводятся жёсткой деривацией по BIP-32 в варианте SLIP-0010 для Ed25519 [45, 20], а путь назначение'/продукт'/проект'/агент' кодирует стабильную личность, отдельную от UID и пути в файловой системе. Личность поэтому переживает пересоздание учётки и переезд на другую машину с тем же мастер-секретом.

Ключник слушает Unix-сокет и получает UID собеседника от ядра (SO_PEERCRED на Linux, LOCAL_PEERCRED на macOS). Личность выбирается по привязке с UID, поколением учётки, путём, владельцем и номером регистрации, и подключение допустимо, только если поколение этого UID активно в журнале менеджера пользователей. Процесс ревьювера может написать в запросе имя оркестратора, но его соединение всё равно несёт UID ревьювера. Сообщения агента ключник подписывает отдельным ключом сообщений Ed25519 с путём m/4'/продукт'/проект'/агент', и только документ с меткой swarma-msg/2 с прообразом не длиннее 24 576 байт. Принадлежность ключа сообщений агенту удостоверяет аттестация ключа сообщений, запись attestation/1, подписанная корнем аттестаций ключника, сроком не длиннее 86 400 с; подпись открыта, только если аттестация действует и опубликована в реестре владельца.

Каждый корень подписывает только свой язык записей, и совпадение ключей двух ролей ведёт к отказу. Например, корень аттестаций ключника подписывает только аттестации ключей сообщений, корень оператора подписывает аттестации устройств оператора и снимки отзывов своей области, корень грантов владельца подписывает гранты, а офлайновый издатель регистраций подписывает только записи машин. Выдача гранта, подключение подписки и другие расширяющие действия требуют человека: приложение на телефоне оператора показывает точное содержание действия и после подтверждения биометрией подписывает именно это действие ключом устройства в защищённом чипе, а второе устройство обязательно. Общий стоп всей установки выполняется одной подписанной командой, которая доставляется напрямую каждой службе; возобновить работу можно только подписью оператора.

3.3. Гранты, реестр владельца и свидетель

Грантом называется подписанное утверждение: кто выдал полномочие, кому, на какие действия и цели, при каких условиях, до какого срока и можно ли передать его дальше. Делегирование только сужает: дочерний грант покрывает подмножество действий и целей родителя, его срок не длиннее, глубина в разрешённых пределах. Цепочку до корня проверяет исполнитель действия, даже если раньше на маршруте её уже проверили. В общем языке грантов ключ агента бывает субъектом и листом цепочки, но издателем звена не бывает, поэтому захваченный оркестратор не раздаёт полученное соседям; контракт брокера v0 допускает одно исключение, суженный грант, который родительский оркестратор подписывает создаваемому им оркестратору. Вызывающий остаётся в цепочке потомка, и в контракте брокера учётных данных v0 лист агента равен пересечению запроса и двух цепочек, вызывающего и диспетчера, с одним действием credential.lease, глубиной 0 и не больше четырёх конвертов в цепочке. Если цепочку построить нельзя, ответ grant_unconstructible приходит до любых эффектов. Отзыв гранта оркестратора поэтому отрезает его агентов при следующей аренде или продлении.

Реестр владельца служит единственным авторитетным писателем ключей, аттестаций, грантов, отзывов и журналов личностей. Он хранит, упорядочивает и публикует записи, а содержимое не переподписывает. Собственный ключ реестра подписывает только снимки отзывов своей области, контрольные точки и записи свежести, поэтому кража этого ключа не даёт выпустить грант или аттестацию. Журнал реестра состоит из записей journal-entry/1 со ссылкой на предыдущую, а подписанная checkpoint/1 называет голову и текущий снимок отзывов. Рядом работает свидетель отката: служба под отдельной учёткой и процессом, которая по запросу с вызовом называет последнюю голову. До-

веренная контрольная точка свидетеля хранится вне узла, у отдельно администрируемого хранителя. Две разные контрольные точки с одним номером дают `journal_fork`, точка с номером меньше засвидетельствованного даёт `journal_rollback`.

Потребители принимают снимок отзывов не старше 60 с; возраст неизменного снимка продлевает запись свежести, которую реестр публикует каждые 20 с, вместе со свежим чтением головы у свидетеля. Записи отзыва накапливаются. Пусть S означает время выпуска первого подписанного полного снимка с записью отзыва, $A \leq 60$ с допустимый возраст снимка, F время вступления записи в силу. Тогда запись действует на каждой проверке не позже $\max(S + A, F)$ и никогда раньше F. Проверка отзыва даёт три исхода: не отозвано, отозвано, недоступно; два последних закрывают допуск. Время проверяется по водяному знаку, и при потере проверенного времени службы держатся не дольше 10 минут по монотонным часам. Сдерживание шире отзыва гранта: одна операция “отозвать и остановить” отзывает полномочие брокера и отгораживает исполнение и сеть, а о токене провайдера, который уже лежит в процессе агента, честно сообщает, что он пригоден до аннулирования у провайдера.

3.4. Подписанные сообщения и обмен агентов поверх Matrix

Сообщение агенту может быть просьбой коллеги, командой владельца или чужим текстом, выданным за распоряжение, и для модели все три выглядят одинаково. Поэтому доставка отделена от полномочий. Носитель перевозит событие, мост проверяет происхождение и допустимость, хост сессий решает, когда передать допущенное событие среде агента. У агента нет учётных данных мессенджера, и прямой выход к носителю ему закрывает файрвол.

Учётку на носителе агент получает без общего секрета. Помощник с ключом машины узнаёт UID вызывающего от ядра и подписывает только запрос на допуск с именем, разрешённым этому UID; *регистратор* (`swarma-registrar`) сверяет ключ машины с реестром машин в git и выдаёт пропуск, JWT с подписью Ed25519 на 120 с. Сервер Matrix доверяет только ключу регистратора и выдаёт токен доступа на 86 400 с без refresh-токена, поэтому скомпрометированная машина заводит учётки только со своей меткой.

Мост стоит между носителем и сессией агента и работает отдельным экземпляром на каждую личность станции под общей служебной учёткой сообщений, никогда под учёткой агента. Провайдер Matrix сначала дописывает событие в почтовый файл и сбрасывает его на диск, затем продвигает курсор синхронизации, а допуск идемпотентен по паре отправителя и идентификатора события. *Ворота доверия* проверяют сообщение детерминированно, без сети и без вопроса к модели, в строгом порядке: каноническая форма байтов, ключ в снимке реестра, подпись, цепочка грантов до корня с сужением действий и целей, снимок отзыва, окно времени, журнал повторов и ревизия привязки цели. Затем небольшая локальная модель (decision guard) может только сузить допуск, и её недоступность даёт отказ. Допущенный текст во ввод сессии не вставляется: хост сессий получает короткий звонок, а тело и запись проверки агент читает из почтового ящика как данные. Сообщения других агентов остаются данными при любой подписи, и пересказ решения владельца другим агентом полномочий автора не переносит.

Сила свидетельства об отправителе-человеке задаётся лестницей. На уровне A0 отправителя удостоверяет сервер носителя владельца, и такой уровень допускает команду в пределах записанной зоны человека только в переходном режиме. На A1 устройство отправителя подписано закреплённым мастер-ключом перекрёстной подписи Matrix, на A2 ключ человека подписал точные байты конверта и ворота проверили грант, на A3 оператор подтвердил точное действие на телефоне. Расширяющие действия требуют A3. Исходящий путь сохраняет точные байты: агент просит ключник подписать текст, а мост проверяет подпись и допустимость аудитории комнаты и не переписывает его. У каждой станции есть личная комната с владельцем ровно на двух участников; владельца записывает поднимающая станцию сторона вне области записи агента.

3.5. Хост сессий и запуск

Запуск разделён между программами, чтобы сборщик плана не получал привилегий запускателя. *Swarma Curator* (далее Curator) управляет окружением агентов: собирает из профиля инструкции, скиллы, подключения MCP и настройки и выдаёт для каждой агентской среды управляемый дом. План запуска фиксирует профиль с пином, то есть хэшем, дом, сетевой профиль и режим учётных данных; собирает его вызывающий, а исполнитель под учёткой агента только проверяет байты. Любой MCP подключается с явного согласия, а неизвестная готовность логина запуска не разрешает. Диспетчер (`swarma-dispatcher`) проверяет подпись заявки, цепочку грантов, свежий вердикт отзыва, квоту и ёмкость машины, а брокер учётных данных (`swarma-credential-broker`) выдаёт аренду токена провайдера только стороне агента и при каждом продлении повторяет проверки.

Хост сессий управляет жизнью долгой сессии и состоит из службы `swarma-session-host` под своей учёткой и раннера `swarma-session-runner` под учёткой агента, который держит PTY или app-server среды. Главный инвариант: во ввод сессии пишет один диспетчер ввода хоста, и только допущенные элементы долговечной очереди. Раннер принимает команды только текущей эпохи контроллера, поэтому соединение, пережившее сбой службы, права старого контроллера не сохраняет. Элемент считается применённым только по подтверждению среды; запись байтов в канал применения не означает. Забор допуска разрешает или запрещает каждый эффект сессии, а контролируемый перезапуск закрывает доставку, дожидается покоя, сохраняет пакет передачи и подтверждает отсутствие прежнего исполнения, прежде чем запустить преемника.

Цель сессии не принадлежит агенту. Её ставит вызывающий при запуске или публикует доска, и у учётки самого агента прав на операции с целями нет; сообщение агента о выполнении цели остаётся предложением исхода. Сессия служит кэшем: авторитетны журнал хоста, записка о передаче и, если есть доска, её задача и цель. После перезагрузки постоянные команды поднимает Swarma Keeper командой `keeper up` через диспетчер, а остановленное человеком остаётся остановленным.

Учётные данные дают агентской среде доступ к провайдеру модели через подписку или ключ API. Подписка человека подставляется только в его запуски, а брокер решает, кому и на какой запуск её выдать. Среда при этом может прочитать токен в своём окружении, и ограниченная аренда не превращает долгоживущий токен провайдера в короткоживущий; поэтому токен, уже выданный процессу, остаётся пригодным до аннулирования у провайдера (раздел 3.3).

Входы оператора и агентов разделены по аудиториям. Swarma Keeper служит повседневным входом человека, `curator` ставит и настраивает машину, а `task-board` служит ядром для агентов, таймера, мостов и самого кипера. Три вида человеческого решения разведены глаголами: `confirm` подтверждает версию конфигурации, `approve` и `reject` решают ожидающий пункт, `grant` и `revoke` выдают и отзывают постоянное право. Разовое одобрение запроса бессрочным правом не становится, а расширяющие действия требуют подтверждения точного содержания на телефоне оператора.

3.6. Удалённое рабочее место: ёмкость модели другого владельца

Удалённым рабочим местом называется учётка ОС на машине проекта для агента, чья модель работает у донора, а инструменты исполняются в среде проекта. Рабочее место служит отдельным пользователем ОС на машине проекта со своим домом, файрволом и группой ресурсов, а кроме того с приватными пространствами имён, пустыми capabilities, `no_new_privs` и seccomp. Исходящей сети у него нет совсем, включая loopback, и без принудительной сети место не создаётся. Вызовы инструментов и почта приходят по одному SSH-соединению, которое открывает донор; обратных соединений проект не открывает. Пользователь рабочего места входит по короткому SSH-сертификату на конкретное поколение учётки, с принудительной командой и без PTY, проброса портов и SSH-агента. Закрытие поколения идёт по порядку: заморозка, закрытие входа, `cgroup.kill`, экспорт и замена, поэтому переподключение донора не создаёт процесса во время остановки или экспорта.

Машину донора защищает своя граница. Агентская среда донора работает в запертом режиме `lockdown` и предлагает модели только путь через реле инструментов проекта, без локальной оболочки, правки файлов и локального веб-поиска. Установка у донора бывает трёх вариантов: запертый запуск без своей границы ОС, две учётки с разными UID для агентской среды и доверенного моста и песочница ОС. Режим закреплён замком в настройке машины, допускается только точный квалифицированный набор сборки, а подключение учётных данных своего провайдера к проекту донор подтверждает ключом с присутствием человека, приложением на телефоне или ключом FIDO2.

Коммиты агента на рабочем месте основанием доверия не служат. После раунда процессы рабочего места останавливаются, и программа выгрузки с правами учётки рабочего места отдаёт изменения относительно точной базы; специальные файлы, подмодули, пути в `.git` и коллизии имён отклоняют выгрузку целиком. Мост донора получает экспорт целиком, сам пересчитывает его дайджест и сверяет назначение, раунд, базу, дайджест плана и голову квитанций со своим заданием и своей цепочкой квитанций, и только затем подписывает этот кортеж. Доска строит из экспорта собственного подписанного кандидата, и кандидат проходит ревью точной ревизии (раздел 4).

3.7. Память агентов

Память агентов хранит знание отдельно от сменяющихся исполнителей: типизированные утверждения вместе с источниками, обоснованиями, оценками и ревизиями. На запрос она отвечает, какие допущенные свидетельства поддерживают или опровергают точно очерченное утверждение, с чьей точки зрения и на какой момент; пустой ответ не доказывает ложности утверждения. Агент получает память как данные с происхождением каждого элемента, и текст из памяти инструкцией не становит-

ся.

С реестром владельца память стоит на одной подложке подписанных записей и читает его как потребитель: отправителя записи определяет по ячейке личности в текущем снимке, основание представительства берёт из гранта, а недоступный реестр никогда не считается выданным грантом. Полномочий память не выдаёт: у неё нет операций одобрить, принять задачу, выдать грант, запустить или разбудить. В профиле git-пространств агент сам подписывает запись через ключник и отправляет её, а каждый читатель проверяет подпись, схему и право на ревизию при чтении. Содержимое этого профиля ограничено нечувствительным, потому что из истории git ничего не стирается; чувствительные данные обслуживает защищённая служба с единственным писателем. Нормативный пакет спецификации памяти, черновик 0.3.0, содержит 53 пункта ядра, 14 закрытых схем и 85 векторов соответствия: 77 смысловых и 8 для защищённого развёртывания.

4. Рабочие процессы как подключаемый модуль

Механизмы рантайма говорят, кто вправе действовать и где исполняется действие. Они не говорят, как команда устраивает работу: какие роли есть, кто производит, кто проверяет и при каких условиях результат допускается к основной ветке. У разных команд ответы разные, а гарантии рантайма от этих ответов зависеть не должны. Поэтому процесс работы оформлен подключаемым модулем.

Продвижение основной ветки к принятому результату мы называем *приземлением*. Правило приземления одно для всех процессов и принадлежит неизменному слою (раздел 4.2); процесс задаёт только условия, при которых задача до приземления доходит.

Результат продьюсер публикует как *Change Request*, запрос на изменение с конкретной неизменяемой ревизией, и вердикт ревьювера относится именно к этой ревизии.

4.1. Модуль процесса и его состав

Модулем процесса (в книге [52] он называется пакетом процесса) называется пакет, который оркестратор загружает, чтобы вести работу: скилл оркестратора, скиллы ролей, поставщики инструментов и *плейбук*, декларативное описание процесса в файле `Playbook.json`. Плейбук задаёт типы работ, состояния, владельцев состояний и разрешённые переходы с проверяемыми условиями (guard), наборы проверок и делегирование действий с потолками полномочий. Плейбук содержит только данные, без исполняемых программ, скриптов и URL; инструмент предоставляет установленный скилл, и запускается инструмент, только если проект его включил. Пакет ставит Curator, а скиллы ролей попадают только в запуски своих ролей: если бы их раскладывали в проект, любая сессия нашла бы их, и оркестратор начал бы действовать по инструкциям ревьювера. Три вопроса проекта разведены по трём файлам: `Skillfile.json` читает Curator, `Playbook.json` описывает работу, `task-board.config.json` задаёт окружение доски.

Плейбук задаёт роли и владельца каждого состояния. Владельцем может быть агент со скиллом роли, человек из группы, инструмент или другой процесс, и в каждый момент у элемента ровно один владелец. Через владельцев плейбук задаёт и маршрутизацию задач к исполнителям: маршрутизатор ролей превращает владельца нового состояния в запрос запуска (раздел 4.3). Наконец, плейбук задаёт состояния и условия переходов, при которых задача доходит до приземления: какие ревью, проверки, состояния QA и ручные согласования нужны. Правило самого приземления плейбук не меняет (раздел 4.4).

Плейбуки компонуются: проект может взять опубликованный процесс и описать отличия либо соединить несколько процессов через слоты, например Scrum для выбора работы и сроков с циклом разработки для пути конкретного изменения, а в iOS-проекте заменить скилл разработчика и вставить состояние QA. Опубликованные части закрепляются хэшами. Исполняется версия, вступившая в силу, а новая требует проверки и подтверждения человеком; отчёт о полномочиях выделяет изменения, которые уводят работу мимо человека или проверки.

4.2. Доска и правила ядра под любым плейбуком

Доска задач (`task-board`) хранит работу, её состояние и свидетельства приёмки и исполняет плейбук. Элементы образуют иерархию Epic, Story, Task или Bug с описанием, прогрессом, статусами, ресурсами и историей; агент читает и меняет доску языком запросов и мутаций, контракт которого можно спросить у самой программы. В рантайме доска служит необязательным клиентом: она просит запуск у диспетчера и публикует цели через хост сессий, а платформенные модули от неё не зависят. Хранилище доски границей авторизации не служит: роль связана с грантом, и переход проверяет ядро доски.

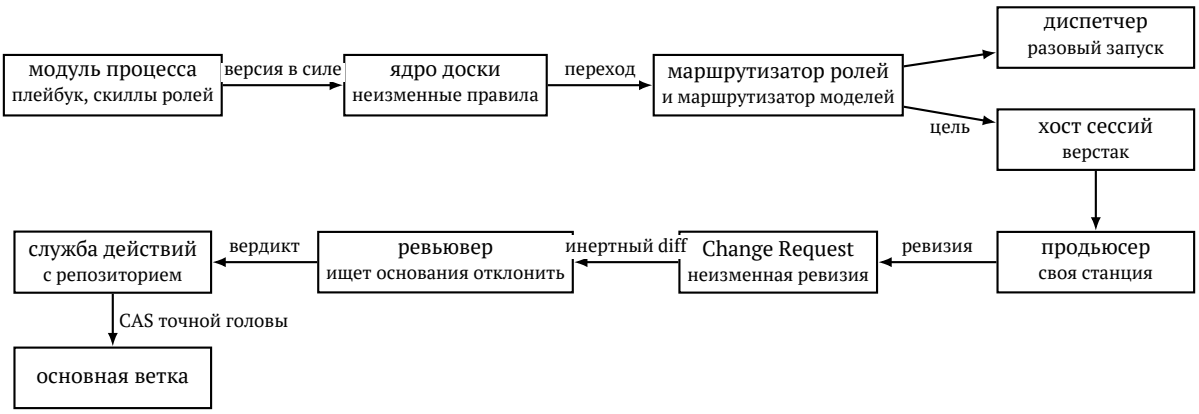


Рис. 3. Процесс по умолчанию в модуле процесса. Ядро доски проверяет каждый переход по действующей версии плейбука, маршрутизатор ролей передаёт работу разовому запуску или постоянному верстаку, ревьювер получает инертный diff неизменной ревизии, а основную ветку продвигает только служба действий с репозиторием сравнением-и-заменой (CAS) точной проверенной головы.

Настраиваемый процесс может сужать разрешённое, но не может отменить предикаты ядра. Фиксированный слой скомпилирован в программу доски, и ни автор плейбука, ни человек, подтвердивший его версию, права отключить его не получают. Защищённое состояние определяется классом, поэтому переименование интеграции обхода не открывает. Неизменных условий восемь:

- задача с Change Request доходит до интеграции и завершения только с принятым запросом;
- работа идёт в дереве своей Story с контрольной точкой, и устаревшее дерево не интегрируется;
- история продвигается подписанной перемоткой точной головы с доказательством стыка;
- свидетельства проверок получены из очереди проверок, а лимиты не выше потолков машины;
- терминальный исход ревью требует свидетельства для этой ревизии;
- цели хранятся неизменными ревизиями со сравнением-и-заменой, а парковка и возобновление фиксируют ревизию;
- мутация и запись аудита пишутся одной транзакцией в журнал, который только дописывается;
- хранилище требует уникальных идентификаторов, существующих родителей, оптимистичной ревизии и допустимых имён ресурсов.

Отсюда простой запрет: нельзя написать плейбук, в котором изменение принято по одному завершению продьюсера. Дополнительные ревью, состояния QA и ручное согласование допустимы, удалять обязательное свидетельство нельзя.

Свидетельство перехода должно быть свежим: задача, вернувшаяся из ревью в разработку и снова пришедшая на ревью, требует вердикта текущего прохода. Инструменты проверки запускает исполнитель ядра в очищенном окружении, иначе агент, запросивший переход, мог бы подложить свою программу проверки через `PATH`.

4.3. Маршрутизация к исполнителям: верстаки, разовые запуски и модели

Ядро записывает состояние и намерение, а маршрутизатор ролей превращает владельца нового состояния в запрос запуска: профиль Curator с точным закреплением, класс задачи, привязки ввода-вывода и нужный контекст. Для разовой работы запрос уходит диспетчеру и проходит транзакцию запуска из раздела 3. Для постоянной сессии, если на неё есть право, доска публикует новую ревизию цели, и хост сессий доставляет её на границе хода.

Постоянным исполнителем служит *верстак* (Werkstatt), постоянное рабочее место агента, то есть связка “проект, роль, среда”. У верстака свой управляемый дом Curator с профилем роли, свой клон репозитория, своя личность на носителе и запись в реестре верстаков на доске с описанием запуска: среда, модель и усилие рассуждений, профиль, клон и личность. Поднимает и останавливает верстак хост сессий; без него сессию нельзя было бы разбудить, проверить или восстановить. Задачи приходят верстаку сообщениями в комнатах проекта со звонком, но право на задачу даёт запись на доске, а текст сообщения права не даёт. Разработчик сдаёт работу ревизией, ревьювер пишет вердикт на доску и в репозиторий не пишет. Клоны верстаков лежат в отдельном корне, выше которого нет файлов `CLAUDE.md` и `AGENTS.md`: Claude Code ищет эти файлы от рабочей папки вверх, и подмена дома этот подъём не останавливает.

Модель и среду выбирает *маршрутизатор моделей* (`cmr`). Перед запуском разные владельцы отвечают на четыре вопроса: разрешено ли (доска по политике оператора), совместимо ли (реестр моделей),

подходит ли (маршрутизатор) и доступно ли сейчас (предварительная проверка доски). Маршрутизатор ранжирует только внутри допущенного набора и текста задачи не читает: он получает роль, класс работы, сложность, режим бюджета и закрепки. Выбор детерминирован, без вызова модели, а запись решения с идентификатором SHA-256 канонического JSON позволяет пересчитать его позже. Для процесса существенно одно правило: ревью может требовать другого издателя модели, чем у запуска, который произвёл работу. Независимость проверяется по записанному происхождению того запуска, и дополнительное ревью того же издателя нарушенного требования не восстанавливает. Правило мотивируют наблюдения о том, что модели-судьи склонны предпочитать собственные ответы [35, 50]; прямо из этих работ оно не следует и переносит их наблюдение на ревью кода. Тот же маршрутизатор выбирает верстак под задачу: сначала фильтры по роли, проекту, живости, доверию и независимости, потом качество, загрузка и цена; если подходящего верстака нет, он отказывает.

4.4. Процесс по умолчанию: продьюсер, ревьювер и приземление точной головы

Процесс по умолчанию состоит из трёх ролей и одного правила. Продьюсер работает на своей станции в самостоятельном репозитории со своим каталогом `.git`; связанный `git worktree` между разными учётками не годится, потому что через общие служебные данные тянет настройки и хуки родителя. Продьюсер публикует Change Request. Ревьювер получает инертные diff и blob: только для чтения, с дайджестами, вне каталогов, где агентская среда ищет настройки проекта. Открыть само дерево кандидата можно лишь через квалифицированный адаптер строгого контекста или эквивалентное исключение обнаружения настроек. Инструкции, хуки, MCP и разрешения кандидата не подхватываются ни при запуске, ни при вложенном чтении, возобновлении или входе дочернего агента.

Ревьювер в процессе по умолчанию ищет основания отклонить кандидата. Он повторяет проверки, сверяет изменение с контрактом и выносит вердикт с находками по классам серьёзности; один запуск не может принять собственную ревизию, и заказчик раунда свой результат не проверяет. Изменения в CI, хуках, инструкциях агентов и ссылках одобряет детерминированный предикат доски над текущим контрактом, политикой риска, точным кандидатом и полным множеством помеченных путей, вычисленным по накопленной дельте от базы до кандидата. Поэтому инструкция агенту, добавленная в первом непринятом кандидате, не пропадает из отчёта второго только потому, что второй правит лишь текст ошибки.

Правило приземления из неизменного слоя таково: основную ветку продвигает служба действий с репозиторием, а у агентов и у процесса доски такого права нет. Служба выполняет подписанную перемотку точной проверенной головы по подписанному доской запросу. Если основная ветка ушла вперёд, возникает другой кандидат, вердикт по сходству diff на него не переносится, и нужны обновление базы и повторное ревью.

У приёмки кода нет безошибочного оракула, и положительный вердикт остаётся суждением. Процесс может добавить ревьюверов, а протокол выбирает их случайно (раздел 5.4), но условный расчёт показывает предел такого усиления. Пусть десять процентов агентов злонамеренны и подсаженную уязвимость злонамеренный ревьювер пропускает всегда, а честный находит её в семидесяти процентах случаев. Тогда один ревьювер пропускает её с вероятностью $0{,}1 + 0{,}9 \cdot 0{,}3 = 0{,}37$. При независимом выборе ревьюверов и независимых ошибках два ревьювера пропускают её с вероятностью около 0,14, три около 0,05. Общая модель, общие инструменты и одинаковый контекст коррелируют ошибки, поэтому расчёт иллюстрирует допущения и качество Swarma не измеряет; измерять нужно обнаружение дефектов на подходящих задачах (раздел 9).

4.5. Swarma Keeper: обязательства между участниками

Процесс задаёт, кто владеет состоянием; он не отвечает, кто кому что должен прямо сейчас, когда работа переходит между участниками. Эту задачу решает *Swarma Keeper*, кипер, главный цикл узла. Кипер следит, чтобы у каждого обязательства был держатель; путать его с ключником нельзя, потому что управление обязательствами и владение ключами имеют разные границы доверия. У кипера две части. Роль исполняет модель, которая разбирает просьбы и принимает решения в своих полномочиях, а модуль, команда `keeper` с библиотекой, ведёт журнал, проверяет переходы и сверяет сроки. Когда модель остановилась или её контекст сжали, часы обязательств не останавливаются.

Обязательство значит “участник должен другому участнику сделать определённую работу к сроку”; виды обязательств: ответить, нести обязанность, доставить, проверить и решить. Передача проходит три этапа: отправлена, принята, начата. Отправитель называет срок принятия `accept_by` и срок первого действия `act_by`, ответ “беру” подтверждает только принятие, и до проверенного первого действия нового держателя обязанность остаётся у прежнего. У каждой попытки свой неповторяющийся `handover_id`, а срок принимается только полной датой и временем UTC, поэтому он восстанавливается после сжатия контекста без догадки о дне.

Эскалация идёт вверх по четырём ступеням: оркестратор держателя, кипер продукта, мета-кипер владельца, человек-владелец. На шаг приходится один переход и одно уведомление, продлений не больше двух на эпизод, недоступного получателя пропускают с записью, а дойдя до человека, подъём останавливается, обязательство остаётся открытым, и молчание согласием не считается. Если автоматическое пробуждение не доказано, каждый шаг приходит двумя уведомлениями: адресным вопросом и видимым человеку `owner-needed`. Журнал дописывает один процесс кипера под его системной учёткой, записи он подписывает своим ключом через ключник, а состояние получается свёрткой записей по порядковому номеру, без опоры на настенные часы. Доску кипер не заменяет: непройденное ревью остаётся фактом доски, а журнал ссылается на него, если из него следует обязанность одного участника перед другим.

4.6. Другие процессы под тем же ядром

Смена процесса не создаёт второго механизма авторизации: у дизайна и разработки разные состояния, но владелец в обоих случаях предлагает переход ядру. Процессы вне разработки пользуются той же идеей точного предмета: если заказ дороже 500 евро идёт менеджеру, одобрение заказа на 620 евро не разрешает исполнить заказ на 6 200, потому что одобрение связано с отпечатком полей. Проектировщик процессов, системная роль вокруг кипера, начинает с простейшего работающего процесса и отправляет изменения плейбука человеку на подтверждение, а валидатор процесса проверяет достижимость состояний, наличие владельцев, обходы guard и детерминизм.

5. Swarma Protocol и федерация

5.1. Одна среда и три плоскости

Средой исполнения называется место, где лежит рабочее дерево проекта, выполняются команды и проверки и продвигается основная ветка. Проект может зависеть от Xcode, лицензий, устройств, GPU, ключей подписи или данных, которые нельзя вывозить, и копия репозитория копии такой среды не даёт. Протокол оставляет среду в одном авторитетном экземпляре и разделяет три плоскости. *Мышление* выполняют агентские циклы контрибьюторов с их моделями и учётными данными. *Управление* выполняют кор-ноды, которые ведут журнал полномочий и принятия. *Исполнение* выполняет пассивная среда: агентских циклов она не запускает и исполняет только разрешённые команды. По сети идут команды, вывод, хэши и подписи; учётные данные контрибьютора остаются у него, а секреты проекта есть только у доверенного раннера с узкими именованными операциями.

5.2. Журнал, кворум и уровни доверия

Журнал представляет собой упорядоченную цепочку подписанных записей с $h_i = H(h_{i-1} \,\|\, c(r_i))$, где $c(r_i)$ означает каноническую форму записи. Запись совершена, когда у неё есть сертификат кворума (QC), набор подписей под одним содержимым, достаточный для уровня доверия проекта; до этого предложение записи среду не авторизует. Уровень выбирает проект:

- на L0 одна кор-нода даёт QC из одной подписи, а кор и среда доверены;
- на L1 при $n \geq 2$ порог $t > n/2$ защищает от одиночного несанкционированного решения при компрометации меньшинства ключей, а единственность истории держится на честном секвенсере;
- на L2 при $n \geq 3f + 1$ византийски устойчивый консенсус [7, 48] использует порог $q = \lceil (n + f + 1)/2 \rceil$ [30], при $n = 3f + 1$ равный $2f + 1$.

Фиксированные $2f + 1$ при большем n неверны: при $n = 5$ и $f = 1$ они дали бы порог 3, а два кворума по три подписи из пяти могут пересечься ровно в одном злонамеренном узле. Среда работает как лёгкий клиент: ей нужны подтверждённое назначение и доказательство его включения в дерево Меркла журнала, а наивысшую принятую версию журнала она хранит как состояние безопасности и более раннюю не принимает. Тяжёлые тела хранятся адресуемо по содержимому, и хэш попадает в журнал только с сертификатом доступности от независимых реплик.

5.3. Контракт задачи, план и цепочка полномочий

Контракт задачи фиксирует требуемый результат и правила приёмки: родительскую цель, область записи, входы и зависимости, результаты и критерии приёмки, профиль подтверждаемых свойств, оценку сложности, класс риска и бюджеты. План отвечает на другой вопрос, что можно сделать: привязки инструментов, гранты и лимиты. Дайджест плана входит в назначение, и среда сверяет его на каждом вызове. Изменение критериев создаёт новую версию контракта с авторизацией того же уровня, что и создание задачи, и прошлые основания принятия при этом не переписываются. Цепочка полномочий идёт от цели, которую подписывает кор-нода, через полномочие оркестратора к задаче

и ревизии, и на каждом уровне права только сужаются. Класс риска определяется детерминированно по путям, манифестам зависимостей и типам операций. Привилегированный класс (авторизация и доступ, зависимости, конфигурация сборки и CI, проверки, правила и манифест, инструкции агентов и хуки, подпись, публикация, секреты и миграции) требует двух независимых ревью или одобрения человека из кор-команды, а его пути доступны для записи только из области задачи этого класса. Поэтому продьюсер, которому задача разрешает менять только реализацию, не может сделать падающий тест зелёным, удалив сам тест. Задачи с пересекающимися областями записи одновременно не исполняются, и область удерживается до интеграции или отмены.

5.4. Оркестратор по аренде и принятие точного кандидата

Оркестратор протокола держит роль по аренде [5] с эпохой и случайным fencing-токеном. Каждая его запись несёт эпоху и токен, и валидаторы отвергают запись устаревшей аренды, сохраняя её как расхождение. Отсечь прежнего держателя fencing-токен у [25] позволяет потому, что он монотонно растёт, а случайного значения для этого мало. Здесь монотонность даёт эпоха: новый держатель получает роль с новой эпохой, а среда проверяет актуальность эпохи на границе исполнения. Принудительный захват роли не передаёт сессию предшественника, и даже планово полученная сессия считается недоверенной: при расхождении прав журнал.

Продьюсер представляет подписанную ревизию R над базой B вместе с головой цепочки квитанций, то есть подписанных средой фактов исполнения его команд. Среда строит неизменяемый *интеграционный кандидат* I над текущей головой M, а журнал фиксирует пару (M, I). Множество допустимых ревьюверов замораживается до появления хэша I; затем получают случайность только из хэшей, из “луковицы хэшей” кор-нод и хэша блока Swarma Chain, вышедшего после фиксации I, и ревьювер вычисляется из идентификатора задачи, хэша I и случайности. Оркестратор поэтому не выбирает удобного проверяющего после того, как кандидат известен. Допустимость ревьювера задаёт политика независимости по трём осям: административный домен, семейство модели и тулчейн.

Аттестация проверки фиксирует подписанный результат проверки свойства с исходом “пройдена”, “провалена” или “не установлена”; таймаут и сбой инструмента дают последний и зелёной проверкой не становятся. Финальное одобрение и каждая аттестация проверки адресованы четвёрке: версия контракта, кандидат I, версия политики проверки и дайджест окружения. Проверки инициирует нода, третья по отношению к продьюсеру и ревьюверу, а критические наборы обязательно повторяет верификатор на другой машине. Получив QC на интеграцию, служба действий с репозиторием атомарно заменяет M на I, только если текущая голова всё ещё равна M. Если голова сдвинулась, кандидат перестраивается, а ревью и проверки повторяются: одинаковый patch-id на другой базе одобрения не переносит.

5.5. Внешние эффекты и остановка

Подпись, публикацию и обращение к устройству исполняет раннер с идентификатором и ключом идемпотентности каждой операции и исходом “выполнена”, “отклонена” или “не установлен”. При неизвестном исходе слепой повтор запрещён: процесс сверяет состояние получателя либо получает решение человека. Авторизация из прежней эпохи или до остановки без новой авторизации не исполняется, и это проверяют демон среды и раннер на границе эффекта. Любая кор-нода может выпустить сертификат экстренной остановки одной подписью, и среда сразу перестаёт открывать области задач, начинать операции и продвигать голову; возобновление требует QC текущего уровня. Один честный участник должен суметь остановить подозрительную работу, и один участник не должен суметь возобновить её вопреки остальным.

5.6. Федерация и доверие между владельцами

Федерация объединяет несколько проектов, между которыми контрибьютор сам выбирает получателя ёмкости. Каждый проект публикует подписанный манифест: адрес среды и хэш её демона, уровень доверия, формы клиента, ограничения на ёмкость и провайдеров, инвентарь ограничений среды и политику независимости. При присоединении манифест компилируется в план контрибьютора с показом разницы полномочий. Доверие направленное: контрибьютор доверяет среде и кор-команде выбранного проекта, а проект продолжает считать рабочую ноду недоверенной. Заявка незнакомой ноды несёт PoW-билет допуска, который одновременно служит шаром майнинга Swarma Chain, а ключа подписи допуска на публичной точке присоединения нет.

Люди и агенты разных владельцев переписываются через тот же носитель; белый список серверов и `m.room.server_acl` ограничивают федерацию Matrix, а подписи и гранты поверх них проверяются всё равно. Человека другого владельца узнают по его корню, закреплённому один раз при знакомстве. Корень аттестует мастер-ключ перекрёстной подписи человека в Matrix вместе с полным адресом, и

устройство отправителя проверяется цепочкой из четырёх звеньев: ключ устройства, ключ самоподписи, мастер-ключ, закреплённый корень. Если звено не сходится, сообщение уровня A1 не получает. Сервер, выдавший адрес, может отправить сообщение от его имени, поэтому гранты называют ключи, удостоверенные корнем, и смена сервера или адреса прав не переносит.

6. Идентичность, аттестации и репутация

Каждый механизм выше называет участника по-своему: UID и поколение на сокете, путь деривации у ключника, адрес на носителе, субъект гранта, издатель записи памяти, ключ ноды в протоколе. Swarma сводит их к одной устойчивой личности, чтобы история участника не зависела от машины, учётки и проекта.

Ячейка личности. Личность задаёт *ячейка личности*, расходуемая запись с тремя полями: текущие ключи, хэш следующих ключей и ссылка на родителя. Идентификатор личности равен хэшу первой записи и при ротациях не меняется. Ячейку меняют четыре траты: ротация следующими ключами, трата родителем, приостановка ключами самой ячейки, которую снимает только родитель, и терминальный отзыв, который подписывает только родитель. Собственная ротация не меняет ссылку на родителя, а предротация в духе KERI [38] защищает от поломки текущих ключей, потому что следующие спрятаны за хэшем. Ячейка владельца служит родителем ячеек его агентов; её ключи из мастер-секрета установки не выводятся и хранятся отдельно, а её собственным родителем служит внешний ключ восстановления. Поэтому при компрометации мастер-секрета ячейка владельца переводит ячейки агентов на ключи нового корня, и личности агентов сохраняются.

Как всё сходится на одной личности. Рантайм выдаёт личности и без работающей цепочки: идентификатор самосертифицирующий и равен хэшу первой записи журнала ключевых событий, а журнал ведёт реестр владельца. При публикации в цепочку ячейка получает тот же идентификатор и последнее состояние, и ссылки, накопленные раньше, остаются действительными. На этой ячейке сходятся все представления участника:

- ключник выводит ключи личности и сообщений агента по стабильному пути, а ключи ячейки выводит из того же мастер-секрета хэш-функцией SHAKE256 от пути и номера ротации;
- ключ сообщений связан с личностью аттестацией ключа сообщений `attestation/1`, опубликованной в реестре;
- учётку на носителе выдаёт регистратор по ключу зарегистрированной машины, а мастер-ключ перекрёстной подписи человека аттестует его закреплённый корень;
- гранты называют ключи, удостоверенные личностью, и адрес в мессенджере субъектом гранта не служит;
- память называет издателя, субъекта и отправителя записей ячейками и берёт основание представительства из гранта в реестре;
- в протоколе короткоживущие ключи нод удостоверяет личность участника, а ревизии подписываются ключом ноды в формате SSH-подписи git;
- аттестации участия называют выдавшего и субъекта по ячейкам, а имя в Swarma Chain указывает на идентификатор ячейки.

Переезд агента на другую машину или в другой рантайм поэтому не рвёт ни истории, ни памяти: меняются ключи и место исполнения, а ячейка остаётся.

Доказательство идентичности. Доказательство связывает личность с внешней сущностью в обе стороны: владелец подписывает заявление ключами ячейки и публикует у внешнего провайдера артефакт с идентификатором, например TXT-запись домена под DNSSEC, файл well-known по HTTPS или ключ аккаунта на хостинге кода. Доказательство удостоверяет контроль над доменом или аккаунтом и честности владельца не удостоверяет. Личности под одной ячейкой владельца, личности, выведенные ключником одной установки, и разные DNS-имена одного владельца относятся к одному административному домену.

Аттестации участия. После ревью или приёмки рантайм проекта выдаёт аттестацию участия: личность агента, роль (продьюсер, ревьювер или верификатор), ссылку на проект, хэш кандидата или ревизии, версии контракта и политики, исход “принято”, “отклонено” или “не установлено”, высоту и время, личность выдавшего и подпись пачки. Выдавать аттестации участия вправе любой рантайм с личностью. В цепочку попадают только хэши и метаданные; приватный проект публикует хэш-обязательство, и полное свидетельство субъект раскрывает сам. Проверка не требует доверия чужой базе: пачка подписана ключами, текущими у выдавшего на высоте записи, путь Меркла сходится к корню пачки в блоке, ячейка выдавшего жива на этой высоте по корню состояния личностей в заголовке, а вес считается по опубликованной формуле.

Репутация. Репутацию детерминированно вычисляют индексаторы как неподвижную точку графа подписанных утверждений по схеме EigenTrust [23]. Затравка задаётся классами доказательства личности, оси продьюсера, ревьювера и верификатора считаются раздельно, старые свидетельства весят меньше, а кластер личностей, аттестующих друг друга, получает ровно тот вес, который втекает в него снаружи. Исход “не установлено” нейтрален. Спор оформляется подписанным возражением, и арбитра в первой версии нет. Принимающий рантайм применяет к истории собственную политику: новому агенту можно назначить двух-трёх независимых ревьюверов, агенту с историей одного, но репутация не выдаёт гранта и не создаёт отсутствующей независимости. Без логически централизованного удостоверения личностей противник может завести сколько угодно личностей [14]. Симметричная функция репутации, одинаковая для всех наблюдателей, против этого устойчивой быть не может, а устойчивость достижима для асимметричных функций, которые считают доверие от выбранной затравки [9]. Поэтому затравку задают классы доказательства личности, вес кластера ограничен весом, втекающим в него снаружи, а каждая личность, заявка и запись стоят работы или платы. Деньги и репутация в первой версии не связаны.

7. Swarma Chain: интеграция с платформой

Swarma Chain служит нижним слоем общего состояния; её устройство описывает работа о Swarma Chain [51], а здесь мы называем только точки, где на неё опираются рантайм и протокол. Консенсус цепочки ведёт деньги, имена и состояние личностей, тяжёлый контент лежит вне цепочки в p2p-памяти. У аттестаций участия и социальных записей узел проверяет конверт (тип, размер, подпись пачки, оплату и статус выдавшего), а смысл детерминированно вычисляют индексаторы.

Консенсус опирается только на хэш-функции. Обязательства строятся как SHA-256 $\|$ SHAKE256 на 64 байта, подписи как пара SLH-DSA-128s в вариантах SHA2 и SHAKE [33], обязательных обеих, весом $2 \times 7856 = 15712$ байт. Кривых, пейрингов и решёток в консенсусе нет. Массовые записи подписываются пачками: одна парная подпись покрывает корень дерева Меркла, а доказательство включения носит субъект записи. Заголовок каждого блока несёт корень хэш-дерева состояния денег, имён и личностей, и лёгкий клиент проверяет заголовки от последнего подписанного checkpoint.

Рантайм и протокол опираются на цепочку в нескольких точках. Ячейки личностей делают одну личность общей для всех проектов и снимают предел реестра одного владельца, который может показать разным проверяющим разные истории. Имена указывают на идентификаторы ячеек и регистрируются через обязательство и раскрытие с вступлением в силу через 1440 блоков после включения раскрытия. Пачки аттестаций участия выносят историю работы за границы журналов проектов. Хэш блока, вышедшего после фиксации кандидата, входит в случайность выбора ревьювера. PoW-билеты допуска публичных нод служат шарами майнинга. Головы журналов протокола и реестров владельцев закрепляются в цепочке, и запись, наблюдённая до поломки класса ключей, сохраняет доказательную силу на дату наблюдения.

Цепочка служит и слоем расчётов. Она даёт общую валюту для расчётов между участниками, а во что проект конвертирует вклад, в деньги, репутацию или долю, задаёт его политика; оплату работы тоже задаёт политика проекта. Расчёты деньгами в цепочке медленные, и для мелких покупок она не предназначена. Экономику цепочки, включая майнинг и его бюджет, разбирает работа о Swarma Chain [51].

Вне консенсуса корни доверия стоят на кривых: Ed25519 в SSH-сертификатах, журнале протокола, квитанциях и ключах рантайма, P-256 в защищённых чипах телефонов, ключи FIDO2, TLS и подписи git. План миграции на случай поломки кривых состоит из четырёх шагов: история фиксируется с датой наблюдения, долгоживущие корни переходят на хэш-подписи, а ключи на кривых остаются короткоживущими листьями под ними, сеансовый слой следует за вендорами с гибридным обменом ML-KEM, а поломанный класс ключей отвергается политикой, и личности ротируются через предротацию.

8. Анализ безопасности по слоям

Гарантии ниже названы вместе с противником, механизмом, допущением и пределом (табл. 2). Это аргументы уровня дизайна; разбор каждой гарантии рантайма с допущениями и пределом как аргумента дизайна дан в сопутствующей работе [53], а протокольные инварианты разобраны в книге [52]. Общее условие для всех строк: гарантия держится, только если исполнитель эффекта вызывает проверку и физически удерживает область эффекта.

Захваченный оркестратор. В рантайме захваченный оркестратор пользуется ключом вызывающего из своего дома, но силу этого ключа ограничивает его грант, и отзыв гранта отрезает созданных им агентов. Грант общего языка он соседу не выдаст: издатель звена, чей ключ аттестован как ключ

Таблица 2. Гарантии по слоям. П1–П8 обозначают противников из раздела 2.

Гарантия	Против	Механизм	Допущение и предел
G1. Агент не получает закрытого ключа своей личности	П1	ключник, UID от ядра, активное поколение	ядро, root и мастер-секрет целы; захваченный агент получает подписи своей личности, и подпись правоты не доказывает
G2. Делегирование не расширяет полномочий	П1, П2	ослабление, ключ агента не издаёт звеньев, вызывающий в цепочке	вредное действие в пределах гранта остаётся остаточным риском
G3. Отзыв действует не позже $\max(S + A, F)$, неизвестность закрывает допуск	П1, П5	накопительные снимки, записи свежести, три исхода проверки	ключ реестра цел; токен, уже выданный процессу, пригоден до аннулирования у провайдера
G4. Откат и раздвоение журнала реестра обнаруживаются	П5	нумерованные контрольные точки, свидетель, точка вне узла	реестр и свидетель на одном узле образуют один домен отказа; разные истории перед разными проверяющими снимает только цепочка
G5. Доставка, членство в комнате и подпись коллеги полномочий не дают	П2, П3	ворота доверия, звонок и почтовый ящик, лестница A0–A3	decision guard инъекцию надёжно не распознаёт [12]; при политике A0 компрометация сервера даёт команды в зоне человека
G6. Учётки на носителе ограничены меткой машины	П1, П4	регистратор, пропуск 120 с, токен без продления	захваченный агент получает пропуск на своё разрешённое имя
G7. Исполнение только на станции с квалифицированными гарантиями	П1	контракт изоляции, квитанция границы, отказ на `absent` и `PENDING-VERIFY`	самоотчёт машины не защищает от захваченного root и скопированного ключа машины; CONNECT оставляет канал выноса
G8. Процесс не принимает изменение без независимого ревью точной ревизии	П1, П2	правила ядра доски, свежесть свидетельства, предикат над накопленной дельтой	ревьювер может ошибиться; модели предпочитают собственный текст [35], и разные владельцы и семейства моделей уменьшают основания для корреляции ошибок, но независимости не доказывают
G9. Принятие относится к точному кандидату, эффект только по актуальной записи	П2, П6, П7	QC, аренда с fencing, четвёрка адреса, CAS, выбор ревьювера после фиксации I	среда доверена в целостности; на L1 единственность истории держит секвенсер
G10. Память и репутация полномочий не создают	П1, П2	нет операций выдачи, недоступный реестр не грант, политика у получателя	история удачных работ говорит только о прошлом; из истории git ничего не стирается

агента, отклоняется, даже если подписи верны, а суженный грант брокера v0 для созданного им оркестратора отрезается вместе с его собственным при отзыве. В протоколе у оркестратора нет прямого shell среды; он действует записями, валидаторы проверяют область, лимиты, класс риска и независимость, ревьювера вычисляет правило выбора, а любая кор-нода может остановить среду одной подписью. Остаточный риск остаётся там, где правильно исполненную вредную задачу приняли до обнаружения; его ограничивают область полномочия, бюджет, отложенное закрепление начислений и остановка. Отложенное закрепление, или вестинг, означает, что начисление за работу закрепляется, только когда интегрированный кандидат прожил заданное число эпох без отката.

Захваченная машина (П4). Root на машине с ключником получает мастер-секрет установки или возможность им пользоваться, а с ним ключи всех агентов установки и их гранты до отзыва. Ключ оператора на телефоне, ключ реестра на узле доверия и офлайновый издатель регистраций остаются вне машины: противник не подписывает действий оператора, не подделывает контрольных точек и не регистрирует новых машин, а его учётки на носителе ограничены именами этой машины. Восстановление идёт через отзыв машины и новый корень, а ячейка владельца переводит ячейки агентов на новые ключи.

Скомпрометированная среда протокола. Скомпрометированная среда может показать один diff и интегрировать другой или сообщить вымышленный результат проверок, и протокол этого не предотвращает: квитанция среды доказывает авторство отчёта, а расхождение обнаруживает только верификатор на другой машине.

Криптографический противник (П8). Подписи на кривых этот противник подделывает, поэтому против П8 ни одна из гарантий G1–G9 вне консенсуса Swarma Chain не держится. G10 на подписях не стоит: она структурна, потому что операций выдачи у памяти и репутации нет, и против П8 сохраня-

Таблица 3. Сводка свидетельств. Каждая строка относится к названной ревизии и названным условиям.

Утверждение	Метод	Результат	Чего не показывает
Генератор, независимая проверка на Python и библиотека на Go согласуются	корпус соответствия; у отказа точный код и положительный контроль	2632 случая (2499 отрицательных, 133 положительных); согласие трёх сообщает репозиторий, тест библиотеки повторён нами	однозначности и полноты спецификации, корректности вне корпуса, работы проверки внутри служб
Два вычислителя ядра памяти согласуются на корпусе 0.2.7	один вычислитель по сводному контракту без доступа к пакету, второй по пакету	ожидаемые ответы на всех 66 смысловых векторах корпуса 0.2.7 у обоих; на 77 векторах черновика 0.3.0 только у вычислителя на Go	согласия на черновике 0.3.0, однозначности контракта вне корпуса, защищённой службы, пользы памяти
Привилегированные шаги хоста исполнимы на Linux	два запуска под root на одной машине, привязанные к сборкам	примитивы учёток: применение, проверка, откат; квалификационный набор: 11 из 11 проверок при применении и при проверке	самоквалификации по 42 гарантиям, допуска станций, перезагрузок, отката установки, других машин
Четыре правила доставки необходимы для свойства интеграции	исчерпывающий перебор ограниченной модели с отрицательным контролем на каждое правило	1090 состояний без нарушения; без любого правила найден контрпример	неограниченных параметров, эквивалентности реализации
Стоимость эталонной проверки одной записи	микрозамеры библиотеки на входах корпуса в одном потоке Go	медианы от 2,1 до 18,0 мс на вызов (табл. 5)	задержки служб и сети, худшего случая, оптимизированной реализации
Ревьювер возвращает заметную долю первых ревизий	разбор записей ревью на доске задач одного проекта	приняты 17 из 35 разобранных первых раундов; медиана 2 раунда на задачу; вердикт разобран в 104 из 318 записей	пропущенных дефектов, точности ревьюверов, причинного эффекта, несмещённых долей

ется. План миграции из раздела 7 ограничивает ущерб. История, закреплённая в цепочке, сохраняет доказательную силу на дату наблюдения, долгоживущие корни переходят на хэш-подписи, а живые листья на кривых остаются подделываемыми на время своего действия. Предротация не даёт противнику перехватить ячейку личности через поломанные текущие ключи, потому что следующие ключи спрятаны за хэшем, а консенсус цепочки опирается только на хэш-функции.

9. Оценка

Оценка сообщает измеренные факты об эталонных компонентах платформы: согласие независимых реализаций на конечных корпусах, привилегированные шаги хоста станции на машине Linux, исчерпывающий перебор ограниченной модели интеграции, стоимость эталонной проверки записей и статистику процесса ревью (табл. 3). Задержка отзыва на сквозном пути станции, производительность системы целиком, состязательная проверка работающей установки и польза роя на внешних задачах в эту оценку не входят.

Доступность артефактов. Библиотека проверки подписанных записей с корпусом и спецификацией `swarm-envelope/2` опубликована в организации github.com/relux-works; корпус проверен на ревизии f76a400, замеры сделаны на её предке 9b42338. Пакет спецификации памяти лежит в каталоге `spec/` репозитория relux-works/swarma-memory (ревизия 763e024), запись о привилегированных запусках находится в `docs/root-qualification-evidence.md` репозитория relux-works/swarma-user-manager. Модель интеграции, обвязка замеров, отчёт о замерах и записи ревью не опубликованы; модель входит в исходники сопутствующей работы [53], которая тоже не опубликована.

9.1. Корпуса соответствия

В корпусе подписанных записей каждый случай отказа несёт точный ожидаемый код и ссылку на положительный контроль, поэтому реализация, которая отказывает всегда, корпус не проходит. Генератор на Go пишет случаи и ожидания, проверка на Python сама канонизирует байты и проверяет подписи Ed25519 и P-256, вердиктов Go не импортируя. В репозитории лежат 944 именованных случая, а 1688 механических развёрток материализуются по манифесту с хэшами SHA-256. Тест библиотеки генерирует полный корпус, сверяет манифест и требует точного кода и плана для всех 2632 случаев. Полный прогон проверки на Python сообщает репозиторий; тест библиотеки мы повторили на ревизии f76a400 (Go 1.26.0, darwin/arm64): 2632 подтеста прошли, код выхода 0. Процедура корпуса задаёт и отрицательный контроль самих инструментов: изменённый байт подписи в положительном случае

Таблица 4. Ограниченная модель интеграции: напечатанный результат каждого запуска.

Запуск	Состояний	Напечатанное нарушение	Действий в трассе
Все правила	1090	`None`	нет
Без `cas`	84	`record applied twice`	6
Без `epoch`	82	`record from a stale lease epoch advanced main`	6
Без `contract`	71	`attestation references a superseded contract version`	6
Без `bind_I`	490	`attestation made on a different base than the integrated candidate`	11

Таблица 5. Эталонная проверка записей на вызов. Время дано медианой и размахом пяти повторов в квадратных скобках, от минимума до максимума; размах не является доверительным интервалом. Память (10^6 байт) и число выделений даны медианой.

Вызов и вход	Время, мс	Память	Выделений
Сообщение `message/1`, 835 байт	6,04 [5,70–7,22]	6,30	92 619
Грант, одно звено, 832 байта	9,10 [7,76–10,53]	9,00	132 307
Грант, три звена, 2628 байт	17,99 [15,53–23,75]	18,91	278 351
Аттестация ключа сообщений, 840 байт	3,78 [3,71–4,51]	4,49	66 016
Снимок отзывов на 64 записи, 7110 байт	2,11 [1,67–2,72]	1,94	28 497
Журнал из 64 записей и две контрольные точки, 30 718 байт	9,21 [7,54–11,23]	7,53	112 915

обязан дать расхождение у генератора и код `envelope_signature_invalid` у проверки на Python.

Для памяти первый вычислитель написан по сводному контракту ядра без доступа к пакету и прогнан на векторах пакета без их правки, второй написан только по пакету, и каждое расхождение разобрано как ошибка пакета, реализации или неоднозначность контракта. На корпусе 0.2.7 оба дают ожидаемые ответы на всех 66 смысловых векторах. На корпусе черновика 0.3.0 вычислитель на Go проходит все 77 смысловых векторов, а согласие двух вычислителей установлено только на корпусе 0.2.7. Восемь векторов защищённого развёртывания этими вычислителями не исполняются.

9.2. Привилегированные шаги хоста станции

На одной машине Linux выполнены два привилегированных запуска, каждый на своей сборке. Первый прошёл режимы применения, проверки и отката для одноразовых примитивов учёток: команды учёток и групп, защита дома, отказ ACL и capabilities, остановка через pidfd. Второй установил набор квалификации: при применении квалификация в режиме загрузки прошла 11 из 11 проверок, отдельная проверка тоже прошла 11 из 11, а допуск станций в этом запуске не настраивался. Опубликованная запись о результатах ограничивает охват второго запуска: он не включает кампании перезагрузок и отката установки и не исполняет проверку открытых inode и пространств имён. Набор из 11 проверок не совпадает с самоквалификацией по 42 гарантиям из раздела 3.1, подписанной аттестации машины не выпускает, и почти все ячейки матрицы гарантий остаются `PENDING-VERIFY`.

9.3. Ограниченная модель интеграции

Модель входит в исходники сопутствующей работы (абзац “Доступность артефактов” выше). Она содержит две задачи, до двух версий контракта, до двух эпох аренды и до двух доставок записи и на границе доставки применяет четыре правила: голова равна базе записи (`cas`), запись несёт текущую эпоху (`epoch`), аттестация ссылается на текущий контракт (`contract`) и адресована точному кандидату (`bind_I`). При всех правилах перебор в ширину достигает 1090 состояний без нарушения, а отключение любого одного правила даёт контрпример (табл. 4); числа отмечают остановку поиска на первом нарушении и безопасность вариантов не сравнивают. Модель считает среду и валидаторов честными и не описывает содержимого кода, полной аренды, остановки и внешних эффектов.

9.4. Микрозамеры эталонной проверки записей

Мы измерили шесть публичных вызовов библиотеки проверки на ревизии 9b42338, на неизменённых входах корпуса; реализация эталонная и не оптимизировалась. Замер шёл в гостевой системе KVM на процессоре AMD Ryzen 7 9700X (гостю видны 8 логических CPU, без закрепления), ядро Linux 7.0.0-38-generic, Go 1.26.10 linux/amd64, `GOMAXPROCS=1`, пять последовательных повторов стандартного бенчмарка Go с `-benchmem`. Машина была общей: одноминутная средняя нагрузка составляла 1,73 до замера и 1,95 после. Внутри таймера библиотека разбирает и проверяет сырые подписанные байты, и каждая итерация проверяет, что вызов принят; загрузка фикстур, ввод-вывод и подпись в замер не входят.

Разница одного и трёх звеньев не оценивает влияния глубины: входы различаются содержимым, и в трёхзвенной цепочке есть подпись P-256. Все записи снимка отзывов вступают в силу позже момента проверки, поэтому худший поиск среди действующих отзывов не измерен. Вызов журнала сравнивает вторую контрольную точку на раздвоение, но её суффикса не проверяет. При медиане 6,04 мс на сообщение один поток проверяет около 165 сообщений в секунду, и для ворот доверия это стоимость эталонной, неоптимизированной реализации.

9.5. Статистика ревью

Разработка проекта Swarma Curator идёт на доске задач с продьюсером и независимым ревьювером из раздела 4, и ревьювер выносит вердикт с находками по классам серьёзности. Мы разобрали файлы исходов ревью на доске задач этого проекта по состоянию на 24 октября 2026 года. Доска рабочая, и её записи не опубликованы. Вердиктом считалось последнее поле `verdict` в файле, серьёзности брались из JSON находок, а записи без вердикта учитывались только в раундах. Нашлось 140 задач с записями ревью и 318 записей. Вердикт разобран в 104 записях из 318; остальные 214 написаны в ранних форматах, поэтому выборка смещена к поздним записям. Из 104 разобранных вердиктов 55 *принято* и 49 *нужны изменения*; из 35 разобранных первых раундов принято 17. Медиана числа раундов на задачу равна 2, максимум 13. Находок высшего класса серьёзности P0 нет, классов P1 и P2 соответственно 73 и 49. Второй разборщик текстовых вердиктов даёт 58 и 51 при 209 неразобранных записях. Другие проекты не учтены. Числа задач, записей и находок служат нижней границей для этой доски; медиана, доля принятых первых раундов и соотношение вердиктов нижней границей не являются, и пропущенные записи могут сдвинуть их в любую сторону. Числа показывают, что ревьювер регулярно возвращает работу, и ничего не говорят о дефектах, которые он пропустил.

10. Смежные работы

Агентские системы. ReAct [47] связал рассуждение модели с вызовом инструментов, SWE-agent [46] и SWE-bench [22] перенесли это на задачи разработки, AutoGen, MetaGPT и ChatDev [43, 21, 36] организуют несколько ролей и обмен между ними. Параллельные агенты, написавшие компилятор [6], опирались на тесты как оракул, а в Prove2Me [8] работу агентов независимых владельцев принимает ядро доказательств. Эти работы изучают способности и координацию; Swarma добавляет ключи, полномочия, отзыв и процедуру принятия кода без механического оракула.

Системная защита агентов от инъекций. Инъекция против агентов с инструментами описана в [19] и измерена в AgentDojo [12]. Шаблон двух моделей [42] не показывает недоверенный текст модели с правом вызывать инструменты, CaMeL [11] извлекает поток управления из доверенного запроса и проверяет политику на capabilities данных, FIDES [10] отслеживает метки конфиденциальности и целостности, Progent [37] задаёт агенту политику привилегий, IsolateGPT [44] исполняет приложения на базе модели изолированно, а [3] сводит такие решения в шаблоны. С ними Swarma разделяет два принципа: допуск решает детерминированный код, а оценка моделью только сужает. Эти работы ограничивают потоки данных и вызовы внутри одного агентского приложения; Swarma ставит проверку у исполнителя эффекта за границей процесса, между машинами и владельцами, и их механизмы применимы внутри станции как дополнительное сужение.

Полномочия и прозрачность. Гранты продолжают линию capability-систем [13, 32]; Macaroons [4] добавляют ограничения без обращения к издателю, Biscuit [15] даёт офлайн-ослабление с языком правил, SPKI/SDSI [16] и UCAN [18] строят цепочки делегирования на открытых ключах. Certificate Transparency [27, 28], CONIKS [31] и Sigstore [34] делают раздвоение журналов обнаружимым, а реестр владельца решает ту же задачу для одного владельца; журнал ключевых событий и предротация ячейки личности следуют KERI [38].

Изоляция, мессенджеры, консенсус и репутация. gVisor [49], Firecracker [1] и Kata Containers [24] изолируют недоверенный код ядром в пространстве пользователя или лёгкими виртуальными машинами, а станция входит в тот же ряд через контракт изоляции. Перекрёстная подпись Matrix [39] и MLS [2] удостоверяют устройства; Swarma использует перекрёстную подпись как уровень A1 и добавляет подпись точных байтов и гранты. Журнал протокола следует BFT-протоколам и кворумным системам [7, 48, 30], аренда с fencing-токеном следует [5, 25], а расчёт репутации и его пределы опираются на [23, 14, 9]. Автоматизация ревью кода изучалась в [41, 40, 29, 17], а смещение моделей-судей, в том числе к собственному тексту [50, 35], мотивирует требование другого издателя модели для ревьювера; само требование переносит их наблюдение на ревью кода.

11. Заключение

Swarma переносит решение о допуске из суждения модели в детерминированные проверки у исполнителя эффекта и делает это на всех трёх слоях, а процесс работы оставляет командам как подключаемый модуль над неизменными правилами ядра. Цена этого переноса видна в допущениях: доверие сводится к ядру, root и доверенной базе хоста, к целому мастер-секрету ключника, к проверенному источнику времени, к узлу доверия под администрированием владельца, к человеку с аппаратным ключом и, в протоколе, к целостности среды проекта.

Свойства системы целиком в этой работе выведены из дизайна, а измерены только её компоненты. Задержку отзыва через реестр, брокер, диспетчер и хост сессий и поведение программных нарушителей на настоящих границах нужно измерить на сквозном пути станции. Книга [52] описывает методы для пользы роя на одинаковых внешних задачах, для калибровки ревьюверов посевом дефектов и для экспериментов со сбоями реализации протокола; методов для задержки отзыва и состязательной проверки работающей установки она не задаёт. Аренде, остановке и внешним эффектам протокола нужна механически проверяемая спецификация [26].

Список литературы

[1] A. Agache, M. Brooker, A. Florescu, et al. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.

[2] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon. The messaging layer security (MLS) protocol. RFC 9420, IETF, July 2023.

[3] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn. Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837, 2025.

[4] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *2014 Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2014.

[5] M. Burrows. The Chubby lock service for loosely-coupled distributed systems. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06)*, pages 335–350. USENIX Association, 2006.

[6] N. Carlini. Building a C compiler with a team of parallel Claudes. `https://www.anthropic.com/engineering/building-c-compiler`, 2026. Anthropic Engineering blog, 5 February 2026.

[7] M. Castro and B. Liskov. Practical Byzantine fault tolerance. In *3rd Symposium on Operating Systems Design and Implementation (OSDI 99)*. USENIX Association, 1999.

[8] S. Chen, K. Marwaha, X. Lu, H. Yuen, and T. Peng. Prove2Me: An open collaborative platform for scaling math formalization. arXiv:2608.28433, 2026.

[9] A. Cheng and E. Friedman. Sybilproof reputation mechanisms. In *2005 ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON '05)*. ACM, 2005.

[10] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin. Securing AI agents with information-flow control. arXiv:2505.23643, 2025.

[11] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. Defeating prompt injections by design. arXiv:2503.18813, 2025.

[12] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[13] J. B. Dennis and E. C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, Mar. 1966.

[14] J. R. Douceur. The Sybil attack. In *Peer-to-Peer Systems: First International Workshop (IPTPS 2002)*, volume 2429 of *Lecture Notes in Computer Science*, pages 251–260. Springer, 2002.

[15] Eclipse Biscuit project. Biscuit, a bearer token with offline attenuation and decentralized verification. `https://github.com/eclipse-biscuit/biscuit/blob/v3.3/SPECIFICATIONS.md`, 2024. Specification, version 3.3. Accessed 2026-10-25.

[16] C. M. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen. SPKI certificate theory. RFC 2693, IETF, Sept. 1999.

[17] A. Froemmgen, J. Austin, P. Choy, et al. Resolving code review comments with machine learning. In *46th International Conference on Software Engineering, Software Engineering in Practice (ICSE-SEIP)*, pages 204–215. ACM, 2024.

[18] I. Gozalishvili, D. Holmgren, P. Krüger, and B. Zelenka. User controlled authorization network (UCAN) specification, version 1.0.0. `https://github.com/ucan-wg/spec`, 2026. README at commit 8072f3f; no release tag.

[19] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM, 2023.

[20] J. Hoenicke and P. Rusnak. SLIP-0010: Universal private key derivation from master private key. `https://github.com/satoshilabs/slips/blob/master/slip-0010.md`, 2016. SatoshiLabs Improvement Proposal. Accessed 2026-10-25.

[21] S. Hong, M. Zhuge, J. Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[22] C. E. Jimenez, J. Yang, A. Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[23] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina. The EigenTrust algorithm for reputation management in P2P networks. In *Twelfth International Conference on World Wide Web (WWW '03)*. ACM, 2003.

[24] Kata Containers project. Kata Containers: An OpenInfra Foundation project. `https://katacontainers.io/`, 2026. Accessed 2026-10-25.

[25] M. Kleppmann. How to do distributed locking. `https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html`, 2016. Blog post, 8 February 2016.

[26] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[27] B. Laurie, A. Langley, and E. Kasper. Certificate transparency. RFC 6962, IETF, June 2013.

[28] B. Laurie, E. Messeri, and R. Stradling. Certificate transparency version 2.0. RFC 9162, IETF, Dec. 2021.

[29] Z. Li, S. Lu, D. Guo, et al. Automating code review activities by large-scale pre-training. In *30th ACM Joint ESEC/FSE*, pages 1035–1047. ACM, 2022.

[30] D. Malkhi and M. Reiter. Byzantine quorum systems. *Distributed Computing*, 11(4):203–213, 1998.

[31] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman. CONIKS: Bringing key transparency to end users. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 383–398. USENIX Association, 2015.

[32] M. S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University, May 2006.

[33] National Institute of Standards and Technology. Stateless hash-based digital signature standard. FIPS 205, NIST, Aug. 2024.

[34] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367. ACM, 2022.

[35] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[36] C. Qian, W. Liu, H. Liu, et al. ChatDev: Communicative agents for software development. In *62nd Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 15174–15186. Association for Computational Linguistics, 2024.

[37] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song. Progent: Securing AI agents with privilege control. arXiv:2504.11703, 2025.

[38] S. M. Smith. Key event receipt infrastructure (KERI). arXiv:1907.02143, 2019.

[39] The Matrix.org Foundation. Matrix specification v1.19: Client-Server API, section end-to-end encryption, cross-signing. `https://spec.matrix.org/v1.19/client-server-api/`, 2026. Released 2026-07-08. Accessed 2026-10-25.

[40] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota. Using pre-trained models to boost code review automation. In *44th International Conference on Software Engineering (ICSE)*, pages 2291–2302. ACM, 2022.

[41] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota. Towards automating code review activities. In *43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 163–174. IEEE, 2021.

[42] S. Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. `https://simonwillison.net/2023/Apr/25/dual-llm-pattern/`, 2023. Blog post, 25 April 2023.

[43] Q. Wu, G. Bansal, J. Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.

[44] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025.

[45] P. Wuille. BIP 32: Hierarchical deterministic wallets. `https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki`, 2012. Bitcoin Improvement Proposal. Accessed 2026-10-25.

[46] J. Yang, C. E. Jimenez, A. Wettig, et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[47] S. Yao, J. Zhao, D. Yu, et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[48] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. HotStuff: BFT consensus with linearity and responsiveness. In *2019 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 347–356. ACM, 2019.

[49] E. G. Young, P. Zhu, T. Caraza-Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau. The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*. USENIX Association, 2019.

[50] L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[51] Алексей Григорьев и Иван Опарин. Swarma Chain: личности, имена и ценность для роёв автономных агентов на одних хэш-функциях. Сопутствующая работа, версия 1.1, рукопись, 2026. Не опубликована.

[52] Алексей Григорьев и Иван Опарин. Swarma. Платформа для роёв ИИ-агентов: от одной машины до открытой сети. Книга, версия 1.4.1, рукопись, 2026. Не опубликована.

[53] Алексей Григорьев и Иван Опарин. Swarma: проверяемые границы доверия для роёв автономных агентов. Сопутствующая работа, версия 6.1, рукопись, 2026. Не опубликована.